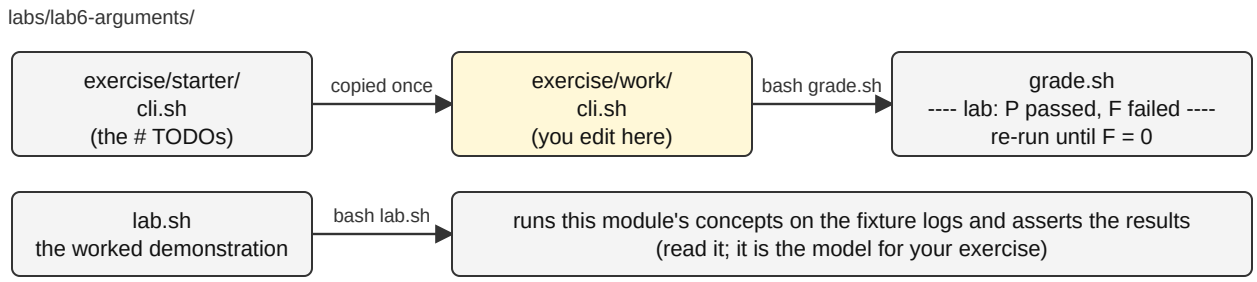
# Lab 6 — Arguments and Options: getopts and File-or-stdin (Instructor Version)

**Instructor Note:** Gray boxes and answers marked in red appear in the instructor copy only.

## Topology




## Environment Table

All paths are relative to `labs/lab6-arguments/exercise/`.

| File | What it is | You edit it? |
|---|---|---|
| `TASK.md` | The short statement of the exercise | No |
| `starter/cli.sh` | `parse_opts` (the loop given as a `# TODO` comment) and `read_input` (prints `FIXME`) | No — the grader copies it |
| `work/cli.sh` | **Your** copy (created by the first `grade.sh` run) | **Yes** |
| `grade.sh` | The grader: 7 checks. It sources your file | No |
| `work/REFLECTION.md` | Your two reflection answers | **Yes** (you create it) |

## Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `parse_opts` — a `getopts` loop with your own error handling.
- **Part 3:** Build `read_input` — read a named file if given, otherwise stdin.
- **Part 4:** Verify with the grader; write the two reflection answers.

## Background / Scenario

A real command line has options (`-n 5`, `-v`), arguments that follow them, and a clear reaction to mistakes: a message on **stderr** and a distinct exit code, **2**, so a caller can tell a usage error from a runtime failure (1). It also reads a file when one is named and stdin otherwise, so it composes in pipelines. This lab gives `logsleuth` exactly that, in two functions:

- `parse_opts [-n N] [-v] ARGS…` — prints `n=<N> v=<0|1> rest=<remaining args>`; `-n` defaults to 10. A missing argument or an unknown option prints an error to stderr and returns 2.
- `read_input [FILE]` — prints FILE if given, otherwise what arrives on stdin.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

## Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M06, steps S06.2–S06.5.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 55 minutes; grading ≈ 5 minutes per learner. Observed: the starter scores `---- lab6: 0 passed, 7 failed ----`; the reference 7/7. Two deviations the grader cannot see, both observed to pass 7/7: `exit 2` instead of `return 2` (the failure checks run in a subshell; in a sourced library `exit` would kill the caller's shell — coach to `return`), and omitting `local OPTIND` (every grader call is a `$( )` substitution, its own subshell, so stale `OPTIND` never leaks — point this out so learners do not conclude the `local` is optional; S06.3 is the proof it is not).

## Part 1: Prepare the exercise folder

### Step 1: Run the grader once.

```
cd labs/lab6-arguments/exercise
bash grade.sh
```

**Expected:** `---- lab6: 0 passed, 7 failed ----`.

### Step 2: Open `work/cli.sh`.

`parse_opts` starts with `local n=10 v=0 OPTIND opt` — keep that line; `OPTIND` must be local so that each call starts parsing from the first argument. The loop is given as a comment. `read_input` prints `FIXME`.

## Part 2: Build `parse_opts`

### Step 1: Write the `getopts` loop.

Replace the comment block and the placeholder `printf` with:

```
parse_opts() {
  local n=10 v=0 OPTIND opt
  while getopts ":n:v" opt; do
    case "$opt" in
      n)  n="$OPTARG" ;;
      v)  v=1 ;;
      :)  printf 'error: -%s needs an argument\n' "$OPTARG" >&2; return 2 ;;
      \?) printf 'error: unknown option -%s\n' "$OPTARG" >&2; return 2 ;;
    esac
  done
  shift $((OPTIND-1))
  printf 'n=%s v=%s rest=%s\n' "$n" "$v" "$*"
}
```

How to read the option string `":n:v"`: the **leading** colon turns on *silent mode* — `getopts` prints no messages of its own and instead reports problems to you: a missing argument sets `opt` to `:`, an

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

unknown option sets `opt` to `?`, and in both cases the offending letter is in `OPTARG`. `n:` means `-n` takes an argument; `v` means `-v` is a flag. After the loop, `shift $((OPTIND-1))` discards the options so `"$*"` holds only the remaining arguments.

**Note:** `getopts` stops at the first argument that is not an option. `parse_opts -n 2 a.log -v` therefore reports `v=0` and `rest=a.log -v` — that is correct behaviour, and the grader checks it.

### Step 2: Try it by hand.

```
source work/cli.sh
parse_opts file.log
parse_opts -n 5 -v a.log
parse_opts -n 2 a.log -v
parse_opts -n; echo "status=$?"
parse_opts -z; echo "status=$?"
```

**Expected:**

```
n=10 v=0 rest=file.log
n=5 v=1 rest=a.log
n=2 v=0 rest=a.log -v
error: -n needs an argument
status=2
error: unknown option -z
status=2
```

## Part 3: Build `read_input`

### Step 1: File if given, stdin otherwise.

```
read_input() {
  if [[ $# -ge 1 ]]; then
    cat -- "$1"
  else
    cat
  fi
}
```

`$#` is the number of arguments. `cat -- "$1"` prints the named file; the `--` ends option parsing so even a file called `-n` is treated as a file. With no argument, a bare `cat` copies stdin to stdout — which is what lets the tool sit in a pipeline.

### Step 2: Try it.

```
printf 'line1\nline2\n' > /tmp/lab6.txt
read_input /tmp/lab6.txt
printf 'piped\n' | read_input
rm -f /tmp/lab6.txt
```

**Expected:** `line1`, `line2`, then `piped`.

## Part 4: Verify and reflect

### Step 1: Run the grader until it is green.

```
bash grade.sh
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

**Expected:** `---- lab6: 7 passed, 0 failed ----`. The checks: defaults when no options (`n=10 v=0 rest=file.log`); `-n 5` and `-v` parsed; options stop at the first non-option; missing argument to `-n` fails with 2; unknown option `-z` fails with 2; `read_input` reads a named file; `read_input` reads stdin.

| Symptom | Cause | Fix |
|---|---|---|
| Bash's own `illegal option -- z` text appears | The leading `:` is missing from the option string | `":n:v"` |
| `missing arg to -n fails (2)` fails although your message prints | The `:` branch prints but does not `return 2` | Add `return 2` |
| `rest=` still shows the options | No `shift $((OPTIND-1))` | Add it after the loop |
| `read_input` prints extra text | A leftover `printf` | `cat` is the whole body |

### Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

**Question 1:** What does the leading `:` in `":n:v"` change about how `getopts` reports errors, and what do your `:` and `\?` branches each receive in `OPTARG`?

> **Answer:** **Met:** the leading `:` switches `getopts` to silent mode — no built-in messages; a missing argument sets `opt` to `:` with the option letter in `OPTARG`, an unknown option sets `opt` to `?` with the letter in `OPTARG`; the script prints its own message and returns 2. **Strong:** notes that without it both cases collapse into `?` with an empty `OPTARG` (observed in S06.2), or that silent mode is what makes the exit code controllable. **Not met:** "the colon means `-n` takes an argument" (that is the *second* colon) or "it enables error handling" with no detail.
>
> *Calibration.* Met: "With `:n:v` getopts keeps quiet and tells me what went wrong through `opt` — `:` for a missing argument, `?` for an unknown option — and puts the letter in `OPTARG`, so I print the message and return 2. Without the first colon bash prints its own text and I cannot control the status." Not met: "The colon after n means -n needs a value." — true, but it answers a different question.

**Question 2:** Why does the course send usage/error text to **stderr** and exit with **2** (not 1) for a usage mistake? Describe one caller that depends on each half of that rule.

> **Answer:** **Met:** stderr keeps the message out of captured output (`out=$(tool …)` must not contain usage text); exit 2 distinguishes a *usage* mistake from a runtime failure (1) so a caller can react differently; names one consumer (`||`, `if`, cron, CI). **Strong:** names a consumer for *each* half (a pipeline consuming stdout for the stderr rule; a wrapper branching on `$?` for the code rule) or cites `logsleuth bruteforce … || alert`. **Not met:** "errors go to stderr by convention" / "2 is the standard" without the consumer.

## Hand-in and grading

Hand in `work/cli.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

**Instructor Note: Grading procedure (≈5 min per learner).**

1. `GRADE_TARGET=/path/to/their/work bash labs/lab6-arguments/exercise/grade.sh` → require `---- lab6: 7 passed, 0 failed ----`.
2. Read `REFLECTION.md` against the descriptors. Record `A-07 PASS` only with 7/7 and both answers at Met or better.
3. **Acceptable variations:** `echo` for the messages (passes). `cat "$1"` without `--` passes (the grader's file has an ordinary name) and fails on `./-n` — mention it in feedback.
4. **Model feedback (Not yet):** "`missing arg to -n fails (2)` FAILED while your parser prints `error: -n needs an argument`: your `:` branch prints but does not `return 2` — the grader checks the *status*." Remediation: S06.2 and S06.5.
5. **Equivalent reassessment:** same grader; Q1′ "Why does `parse_opts -n 2 a.log -v` report `v=0`?" (Met: getopts stops at the first non-option) and Q2′ "Why `cat -- "$1"` rather than `cat "$1"`?" (Met: `--` ends option parsing so a file named `-n` is a file).

**Instructor Note: Appendix — reference solution (`solution/cli.sh`, observed 7/7):**

```
#!/usr/bin/env bash
# Lab 6 solution — getopts option parsing and file-or-stdin input.

parse_opts() {
  local n=10 v=0 OPTIND opt
  while getopts ":n:v" opt; do
    case "$opt" in
      n)  n="$OPTARG" ;;
      v)  v=1 ;;
      :)  printf 'error: -%s needs an argument\n' "$OPTARG" >&2; return 2 ;;
      \?) printf 'error: unknown option -%s\n' "$OPTARG" >&2; return 2 ;;
    esac
  done
  shift $((OPTIND-1))
  printf 'n=%s v=%s rest=%s\n' "$n" "$v" "$*"
}

read_input() {
  if [[ $# -ge 1 ]]; then
    cat -- "$1"
  else
    cat
  fi
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.